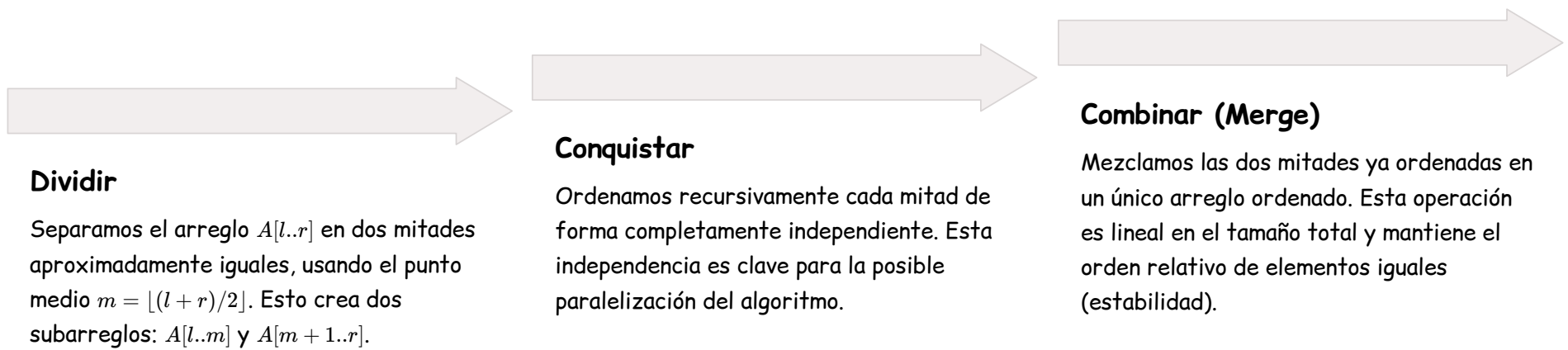

Algoritmo III

Mergesort: Ordenación Eficiente

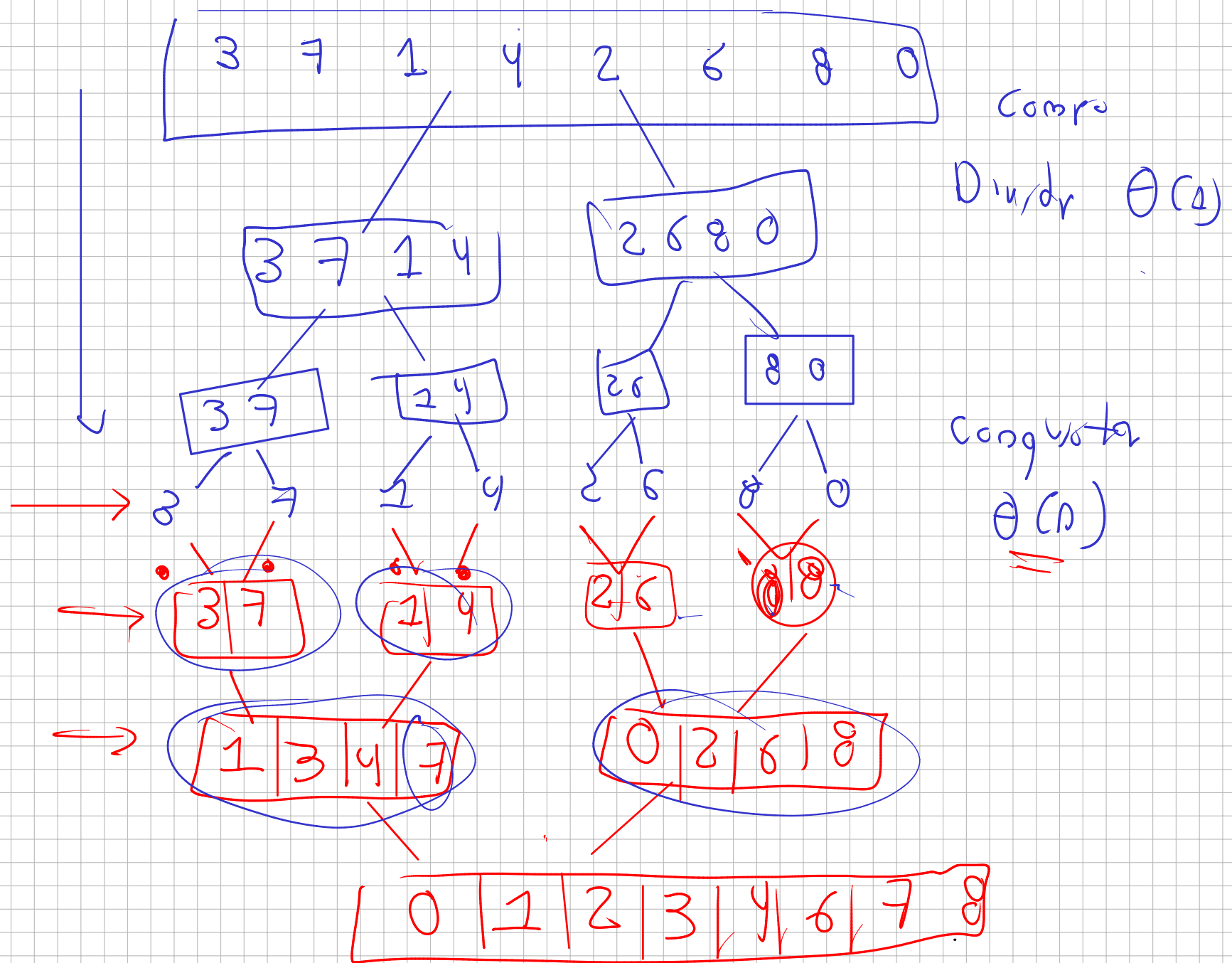
Mergesort — Idea y Proceso

Mergesort es un algoritmo de ordenación que ejemplifica perfectamente la elegancia y eficiencia del paradigma Divide y Vencerás. Fue diseñado por John von Neumann en 1945 y sigue siendo uno de los algoritmos de ordenación más importantes.

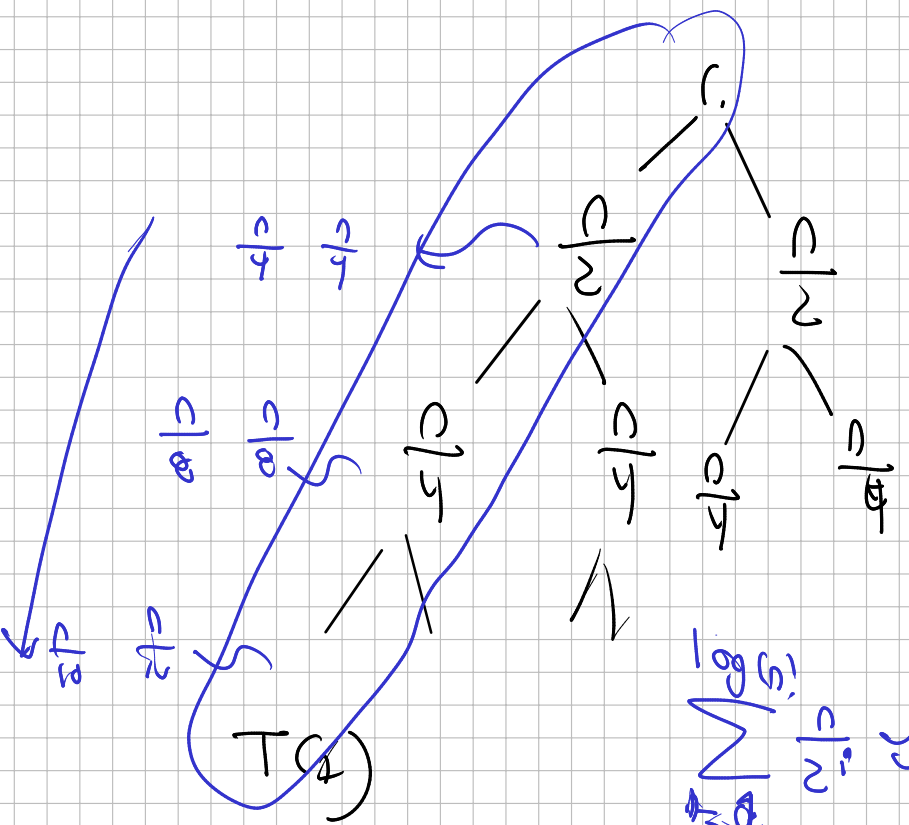


El **caso base** es fundamental: un subarreglo de tamaño 0 o 1 ya está ordenado por definición, por lo que no requiere ningún procesamiento. La fase de merge es el corazón del algoritmo: dados dos subarreglos ordenados, los combinamos en tiempo lineal comparando siempre los primeros elementos no procesados de cada subarreglo.

- ❑ **Propiedad importante:** Mergesort es un algoritmo *estable*, lo que significa que preserva el orden relativo de elementos con claves iguales. Esta característica es crucial en aplicaciones donde los datos tienen múltiples campos de ordenación.

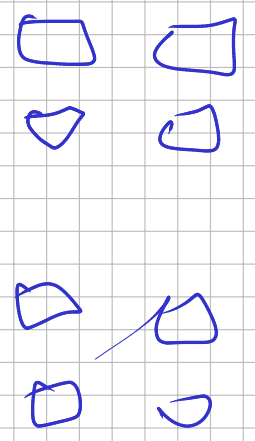


$$T(n) = 2T\left(\frac{n}{2}\right) + \underbrace{\Theta(1) + \Theta(n)}_{\Theta(n)}$$



$\sum_{i=1}^n \frac{c_i}{2}$
 $\sum_{i=1}^n \frac{c_i}{2}$
 $\sum_{i=1}^n \frac{c_i}{2}$

$\frac{1}{2} \log$
 $\frac{1}{2} \log$
 $\frac{1}{2} \log$
 $\frac{1}{2} \log$



Mergesort — Análisis y Complejidad

La recurrencia de Mergesort refleja su estructura de dos llamadas recursivas sobre mitades del problema, más el costo lineal de mezclar:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1)$$

Aplicación del Teorema Maestro

Identificamos los parámetros:

- $a = 2$: dos llamadas recursivas (mitades izquierda y derecha)
- $b = 2$: dividimos el problema por la mitad
- $f(n) = \Theta(n)$: costo de la operación merge

Calculamos el exponente crítico:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Observamos que $f(n) = \Theta(n) = \Theta(n^1 \cdot \log^0 n)$, por lo que aplicamos el **Caso 2** del Teorema Maestro con $k = 0$:

$$T(n) = \Theta(n^1 \cdot \log^{0+1} n) = \Theta(n \log n)$$

La garantía de $\Theta(n \log n)$ en todos los casos hace a Mergesort ideal para aplicaciones donde el peor caso importa (sistemas en tiempo real, bases de datos críticas).

Complejidades Resultantes

Complejidad Temporal:

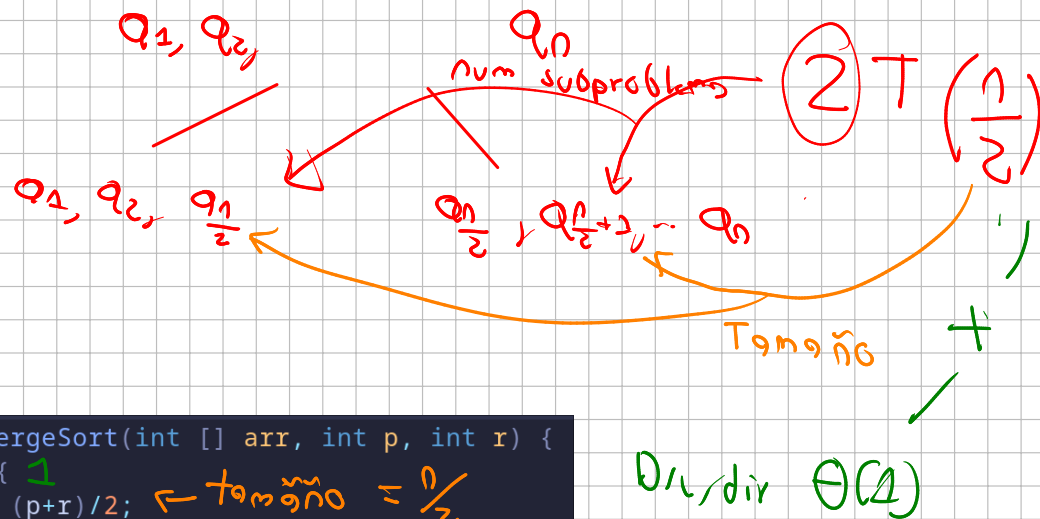
$$\Theta(n \log n)$$

Esta complejidad se mantiene en el *mejor*, *promedio* y *peor* caso. A diferencia de Quicksort, Mergesort siempre divide el problema por la mitad, sin importar los datos de entrada.

Complejidad Espacial:

$$\Theta(n)$$

Requiere espacio adicional para los arreglos temporales durante la operación merge. Este es el principal inconveniente frente a algoritmos in-place.



```

public void mergeSort(int [] arr, int p, int r) {
    if (p < r) {
        int q = (p+r)/2;
        mergeSort(arr, p, q);
        mergeSort(arr, q+1, r);
        merge(arr, p, q, r);
    }
}

```

costo combinar

El proceso de dividir llega hasta el CASO BASE

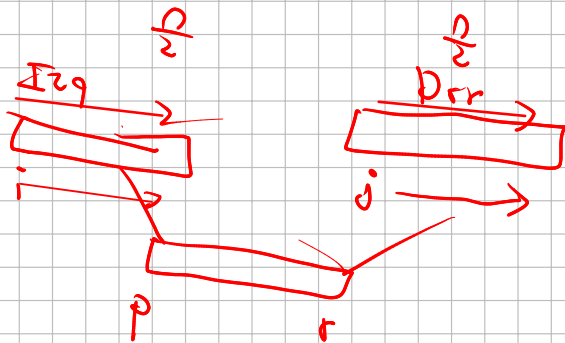
Que el arreglo tenga tamaño 1, $p = r$

Dado que el arreglo ya esta ordenado NO REALIZAMOS OPERACION

Combinar



Invariante, es que tenemos dos arreglos izq, der que están ordenados



1) Comparación $\Theta(n)$



$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1) + \Theta(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

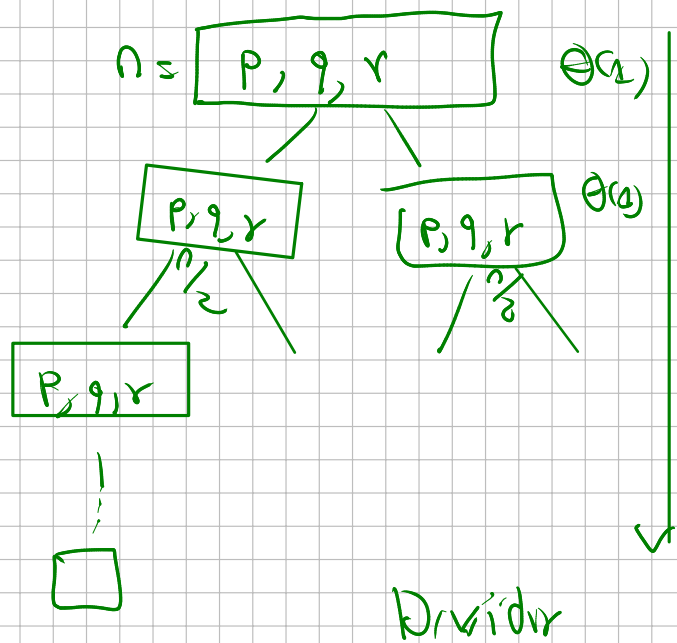
$$\Theta(\log^{k+1} n \times n^{\log_b a}) = \left(\frac{1}{2}\right)(\log n) \times n$$

$$\log_b 2 = \log_2 2$$

$$1) f(n) \approx O(n^{\log_b a - \epsilon}) \quad \epsilon > 0$$

$$\Theta(n) \approx O(n^{2-\epsilon})$$

$$2) \Theta(n) \approx \Theta(n^{\log_b a}) \quad k \geq 0$$



n
 $n/2$
 $n/4$
 $n/8$
 1

0

1

2

3

$\dots$

n
 $n/2$
 $n/4$
 $n/8$
 $n/16$

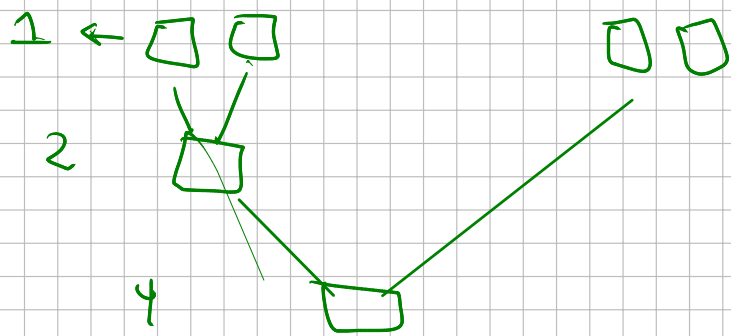
$$2^k = 1$$

$$n = 2^k$$

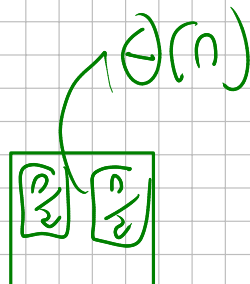
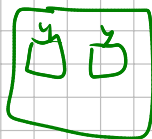
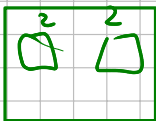
$$\log(n) = \log(2^k)$$

$$\log(n) = k$$

$$\Theta(\log(n))$$



$\infty \dots \frac{c}{2} \dots c$



Mergesort — Pseudocódigo del Merge

La operación **merge** es el corazón de Mergesort. Recibe un arreglo con dos segmentos ordenados contiguos y los fusiona en un único segmento ordenado:

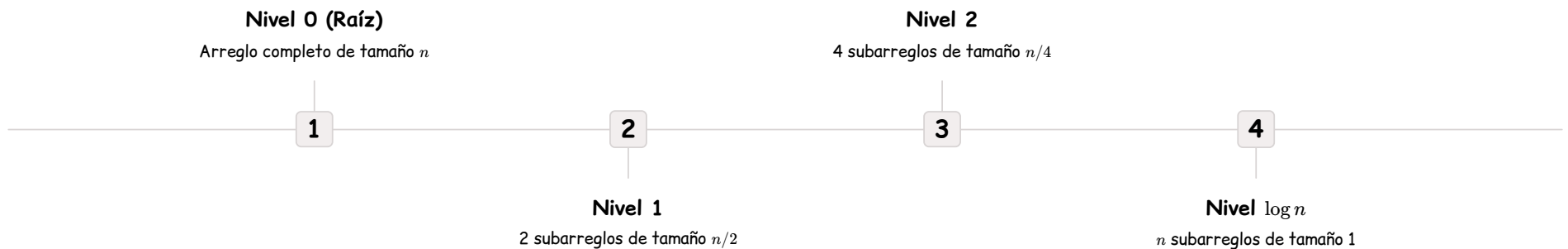
```
procedimiento merge(A, l, m, r):  
  // Crear copias de las dos mitades ordenadas  
  L ← copia de A[l..m]    // Mitad izquierda  
  R ← copia de A[m+1..r]  // Mitad derecha  
  
  // Índices para recorrer L, R y A  
  i ← 0                    // Índice en L  
  j ← 0                    // Índice en R  
  k ← l                    // Índice en A (destino)  
  
  // Mezclar comparando elementos de L y R  
  mientras i < |L| y j < |R|:  
    si L[i] ≤ R[j]:  
      A[k] ← L[i]  
      i ← i + 1  
    en otro caso:  
      A[k] ← R[j]  
      j ← j + 1  
    k ← k + 1  
  
  // Copiar elementos restantes de L (si existen)  
  mientras i < |L|:  
    A[k] ← L[i]  
    i ← i + 1  
    k ← k + 1  
  
  // Copiar elementos restantes de R (si existen)  
  mientras j < |R|:  
    A[k] ← R[j]  
    j ← j + 1  
    k ← k + 1
```

❏ **Nota sobre estabilidad:** El uso de `≤` en lugar de `<` en la comparación es crucial para mantener la estabilidad del algoritmo. Cuando dos elementos son iguales, siempre tomamos el de la mitad izquierda primero.

Mergesort — Pseudocódigo Completo

El procedimiento principal de Mergesort coordina la recursión y llama a merge:

```
procedimiento mergesort(A, l, r):  
  // Caso base: un elemento o menos ya está ordenado  
  si  $l \geq r$ :  
    retornar  
  
  // Dividir: calcular punto medio  
   $m \leftarrow \lfloor (l + r) / 2 \rfloor$   
  
  // Conquistar: ordenar recursivamente ambas mitades  
  mergesort(A, l, m)  // Ordenar mitad izquierda  
  mergesort(A, m+1, r) // Ordenar mitad derecha  
  
  // Combinar: mezclar las mitades ordenadas  
  merge(A, l, m, r)  
  
// Llamada inicial para ordenar todo el arreglo  
mergesort(A, 0, n(A) - 1)
```



El árbol de recursión tiene altura $\Theta(\log n)$ y en cada nivel se realiza trabajo $\Theta(n)$ (suma de todos los merges en ese nivel), lo que confirma la complejidad total $\Theta(n \log n)$.

Mergesort — Implementación en Python

Función Merge

```
def merge(left, right):
    """
    Mezcla dos listas ordenadas en una lista ordenada.

    Args:
        left: lista ordenada (mitad izquierda)
        right: lista ordenada (mitad derecha)

    Returns:
        Lista ordenada con todos los elementos
    """
    i = j = 0
    out = []

    # Mezclar comparando elementos
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1

    # Agregar elementos restantes
    out.extend(left[i:])
    out.extend(right[j:])

    return out
```

Función Mergesort

```
def merge_sort(a):
    """
    Ordena una lista usando Mergesort.

    Args:
        a: lista de elementos comparables

    Returns:
        Nueva lista ordenada
    """
    # Caso base: lista vacía o un elemento
    if len(a) <= 1:
        return a

    # Dividir por la mitad
    mid = len(a) // 2

    # Conquistar: ordenar recursivamente
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    # Combinar: mezclar resultados
    return merge(left, right)

# Ejemplo de uso
datos = [64, 34, 25, 12, 22, 11, 90]
ordenados = merge_sort(datos)
print(f"Arreglo ordenado: {ordenados}")
```

❏ **Observación:** Esta implementación en Python crea nuevas listas en cada llamada (no es in-place), lo que es elegante pero usa más memoria. Existe una versión in-place que modifica el arreglo original y usa espacio auxiliar solo para el merge.

Algoritmo IV

Quicksort: Ordenación por Partición

Quicksort — Idea y Proceso

Quicksort es un algoritmo de ordenación altamente eficiente, desarrollado por Tony Hoare en 1960. Se basa en el paradigma "divide y vencerás", similar a Mergesort, pero su estrategia de división es diferente y se realiza "in-place" en muchos casos. Es uno de los algoritmos de ordenación más utilizados debido a su excelente rendimiento promedio.

El proceso de Quicksort se resume en tres pasos fundamentales que se aplican recursivamente:



Seleccionar un Pivote

Se elige un elemento del arreglo, conocido como pivote. La elección del pivote es crucial para el rendimiento del algoritmo.



Particionar el Arreglo

Se reorganiza el arreglo de tal manera que todos los elementos menores que el pivote se mueven a su izquierda, y todos los mayores se mueven a su derecha.



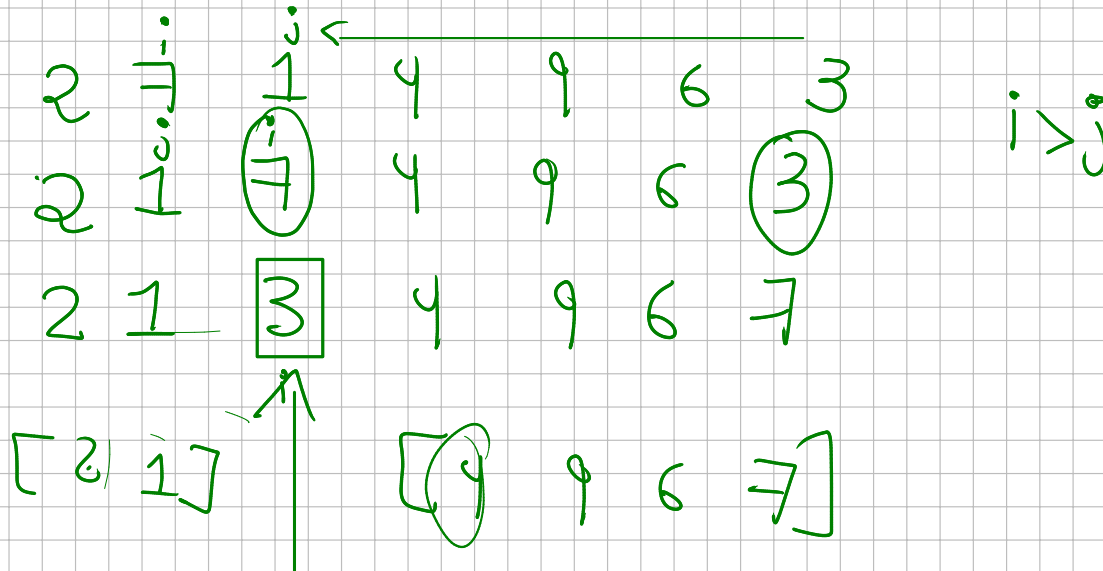
Ordenar Recursivamente

Se aplican los pasos anteriores de forma recursiva a los dos sub-arreglos resultantes (elementos menores y elementos mayores que el pivote).



$arr[i] < 3$
 $arr[j] \geq 3$

El i busca los $\geq$ pivote, j busca los que sean menores, cuando encuentra una match hace el intercambio



Quicksort — Análisis y Complejidad

El rendimiento de Quicksort, al igual que Mergesort, se analiza a través de relaciones de recurrencia. Sin embargo, la estrategia de partición de Quicksort introduce una variabilidad significativa que afecta su complejidad, dependiendo en gran medida de la elección del pivote.

Caso Mejor y Promedio

El escenario ideal y el más frecuente en la práctica es cuando el pivote divide el arreglo de manera equilibrada, creando dos subproblemas de tamaño similar. Esto ocurre típicamente con una buena selección de pivote o, en promedio, con una selección aleatoria.

La relación de recurrencia para este caso es:

$$T(n) = 2T(n/2) + O(n)$$

Donde $2T(n/2)$ representa las dos llamadas recursivas a subarreglos de la mitad del tamaño, y $O(n)$ es el tiempo para la operación de partición (que recorre el arreglo una vez). La solución a esta recurrencia es $O(n \log n)$.

Caso Peor

El caso peor ocurre cuando el pivote divide el arreglo de forma muy desequilibrada, por ejemplo, cuando siempre se selecciona el elemento más pequeño o el más grande como pivote. Esto da como resultado que un subarreglo esté vacío o tenga un solo elemento, y el otro contenga casi todos los demás elementos.

La relación de recurrencia en este escenario es:

$$T(n) = T(n - 1) + O(n)$$

Aquí, $T(n - 1)$ es la llamada recursiva a un subarreglo de tamaño $n - 1$, y $O(n)$ sigue siendo el costo de la partición. La solución a esta recurrencia es $O(n^2)$.

A pesar de un posible caso peor de $O(n^2)$, en la práctica, Quicksort es extremadamente rápido. Esto se debe a que, si el pivote se elige aleatoriamente, la probabilidad de caer repetidamente en el caso peor es muy baja, y el tiempo esperado de ejecución es siempre $O(n \log n)$. La complejidad de espacio para Quicksort es $O(\log n)$ en el caso promedio y $O(n)$ en el peor caso (para la pila de recursión), aunque implementaciones in-place pueden optimizar esto.

Quicksort — Pseudocódigo de Partición

El corazón del algoritmo Quicksort reside en la función de partición, que se encarga de reordenar un subarreglo alrededor de un elemento pivote. Su objetivo es colocar el pivote en su posición final ordenada, de tal manera que todos los elementos menores o iguales a él estén a su izquierda y todos los mayores estén a su derecha.

```
// Función que particiona un subarreglo A[l...r]
// y devuelve la posición final del pivote.
PARTITION(A, l, r)
    // 1. Elegir el pivote. En esta implementación, se selecciona
    // el último elemento del subarreglo como pivote.
    pivot = A[r]

    // 2. Inicializar el índice 'i'. Este índice se encargará de
    // rastrear el final de la sección de elementos menores o
    // iguales al pivote.
    i = l - 1

    // 3. Iterar a través del subarreglo desde 'l' hasta 'r-1'
    // usando el índice 'j'.
    for j = l to r - 1
        // Si el elemento actual A[j] es menor o igual que el pivote,
        // significa que debe ir a la izquierda del pivote.
        if A[j] <= pivot
            // Incrementar 'i' para extender la sección de
            // elementos menores.
            i = i + 1
            // Intercambiar A[i] y A[j]. Esto mueve A[j] a su
            // posición correcta en la sección de menores.
            swap(A[i], A[j])

    // 4. Una vez que todos los elementos han sido procesados,
    // colocar el pivote en su posición final. El pivote debe
    // ir justo después del último elemento de la sección de menores.
    swap(A[i + 1], A[r])

    // 5. Devolver la posición final del pivote.
    return i + 1
```

Quicksort — Pseudocódigo Completo

El algoritmo completo de Quicksort combina la función de partición (vista previamente) con llamadas recursivas para ordenar los subarreglos resultantes. La eficiencia de Quicksort radica en esta división sistemática del problema hasta que cada subarreglo es lo suficientemente pequeño como para ser considerado ordenado.

```
// Función principal de Quicksort
QUICKSORT(A, l, r)
    // Caso base de la recursión:
    // Si el subarreglo tiene 0 o 1 elementos, ya está ordenado.
    if l < r
        // 1. Particionar el arreglo:
        // 'pi' es el índice de partición, A[pi] ya está en su lugar correcto.
        pi = PARTITION(A, l, r)

        // 2. Ordenar recursivamente el subarreglo izquierdo:
        // Elementos menores que el pivote.
        QUICKSORT(A, l, pi - 1)

        // 3. Ordenar recursivamente el subarreglo derecho:
        // Elementos mayores que el pivote.
        QUICKSORT(A, pi + 1, r)

// PARTITION(A, l, r) se implementa como en el pseudocódigo anterior.
// Es crucial para el funcionamiento de QUICKSORT.
```

Quicksort — Implementación en Python

A continuación, presentamos la implementación del algoritmo Quicksort en Python. Dividimos el código en la función de partición (que reordena los elementos alrededor del pivote) y la función principal de Quicksort (que maneja la recursión).

Función PARTITION()

```
def partition(arr, low, high):
    """
    Particiona el arreglo 'arr' alrededor de un pivote, colocando
    todos los elementos menores o iguales al pivote a su izquierda,
    y todos los mayores a su derecha. El pivote es el último elemento.

    Args:
        arr (list): La lista a particionar.
        low (int): El índice inicial del subarreglo.
        high (int): El índice final del subarreglo (donde está el pivote).

    Returns:
        int: El índice final del pivote después de la partición.
    """
    # Elegimos el último elemento como pivote.
    pivot = arr[high]

    # 'i' es el índice del elemento más pequeño, que indica
    # la posición donde debería ir el siguiente elemento menor que el pivote.
    i = low - 1

    # Iteramos a través del subarreglo desde 'low' hasta 'high - 1'.
    for j in range(low, high):
        # Si el elemento actual es menor o igual que el pivote.
        if arr[j] <= pivot:
            # Incrementamos 'i' y realizamos un intercambio para
            # mover el elemento actual a la parte "menor".
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    # Finalmente, colocamos el pivote en su posición correcta.
    # Está justo después del último elemento menor que él.
    arr[i + 1], arr[high] = arr[high], arr[i + 1]

    # Devolvemos la posición final del pivote.
    return i + 1
```

Función QUICKSORT()

```
def quicksort(arr, low, high):
    """
    Implementación del algoritmo de ordenamiento Quicksort.
    Ordena el subarreglo 'arr[low...high]' utilizando
    el enfoque de dividir y conquistar.

    Args:
        arr (list): La lista a ordenar.
        low (int): El índice inicial del subarreglo a ordenar.
        high (int): El índice final del subarreglo a ordenar.
    """
    # Condición base para la recursión:
    # Si 'low' es menor que 'high', significa que hay al menos
    # dos elementos en el subarreglo para ordenar.
    if low < high:
        # 'pi' (partition index) es el índice donde el pivote
        # se encuentra después de la partición. En este punto,
        # arr[pi] ya está en su posición final ordenada.
        pi = partition(arr, low, high)

        # Llamadas recursivas para ordenar los dos subarreglos:
        # 1. El subarreglo de elementos menores que el pivote.
        quicksort(arr, low, pi - 1)

        # 2. El subarreglo de elementos mayores que el pivote.
        quicksort(arr, pi + 1, high)

    # Ejemplo de uso:
    # mi_lista = [10, 7, 8, 9, 1, 5]
    # n = len(mi_lista)
    # quicksort(mi_lista, 0, n - 1)
    # print("Lista ordenada:", mi_lista)
```

Ejercicios Propuestos

Para consolidar su comprensión del paradigma "Divide y Vencerás", le presentamos una serie de desafíos que aplican sus principios. Estos ejercicios están diseñados para poner a prueba su capacidad de aplicar la recursión y la eficiencia.

1 Encontrar el k-ésimo elemento más pequeño
Implemente un algoritmo eficiente, similar a Quicksort, para encontrar el k-ésimo elemento más pequeño en un arreglo desordenado. No es necesario ordenar el arreglo completo.

2 Contar Inversiones en un Arreglo
Dado un arreglo, cuente el número de pares $(A[i], A[j])$ tales que $i < j$ y $A[i] > A[j]$. Desarrolle una solución con una complejidad de tiempo óptima utilizando "Divide y Vencerás".

3 Par de Puntos Más Cercano
Dados n puntos en un plano 2D, encuentre el par de puntos con la distancia euclidiana más pequeña. Desafíese a lograr una solución más eficiente que la aproximación de fuerza bruta.

4 Multiplicación de Matrices de Strassen
Implemente el algoritmo de Strassen para la multiplicación de matrices, que es más eficiente asintóticamente que el algoritmo estándar, especialmente para matrices grandes. Enfóquese en la recursión.

5 Mediana de Dos Arreglos Ordenados
Dados dos arreglos ordenados de tamaño m y n respectivamente, encuentre la mediana de los dos arreglos combinados. La complejidad de tiempo esperada es $O(\log(m+n))$.

6 Implementar Quicksort Aleatorizado
Modifique su implementación de Quicksort para seleccionar el pivote de forma aleatoria en cada partición. Analice cómo esta variación afecta el rendimiento en los casos promedio y peor.

Conclusiones: El Poder de "Divide y Vencerás"

El paradigma "Divide y Vencerás" es una estrategia algorítmica fundamental que hemos explorado a fondo. Sus principios son esenciales para el diseño de soluciones eficientes a problemas complejos.

1 Paradigma Fundamental

Representa un enfoque poderoso para el diseño de algoritmos, transformando problemas intrincados en conjuntos de tareas más sencillas y manejables.

2 Descomposición Recursiva

La esencia es la capacidad de dividir un problema original en subproblemas más pequeños y auto-similares, resolviendo cada uno de forma independiente antes de combinarlos.

3 Eficiencia Comprobada

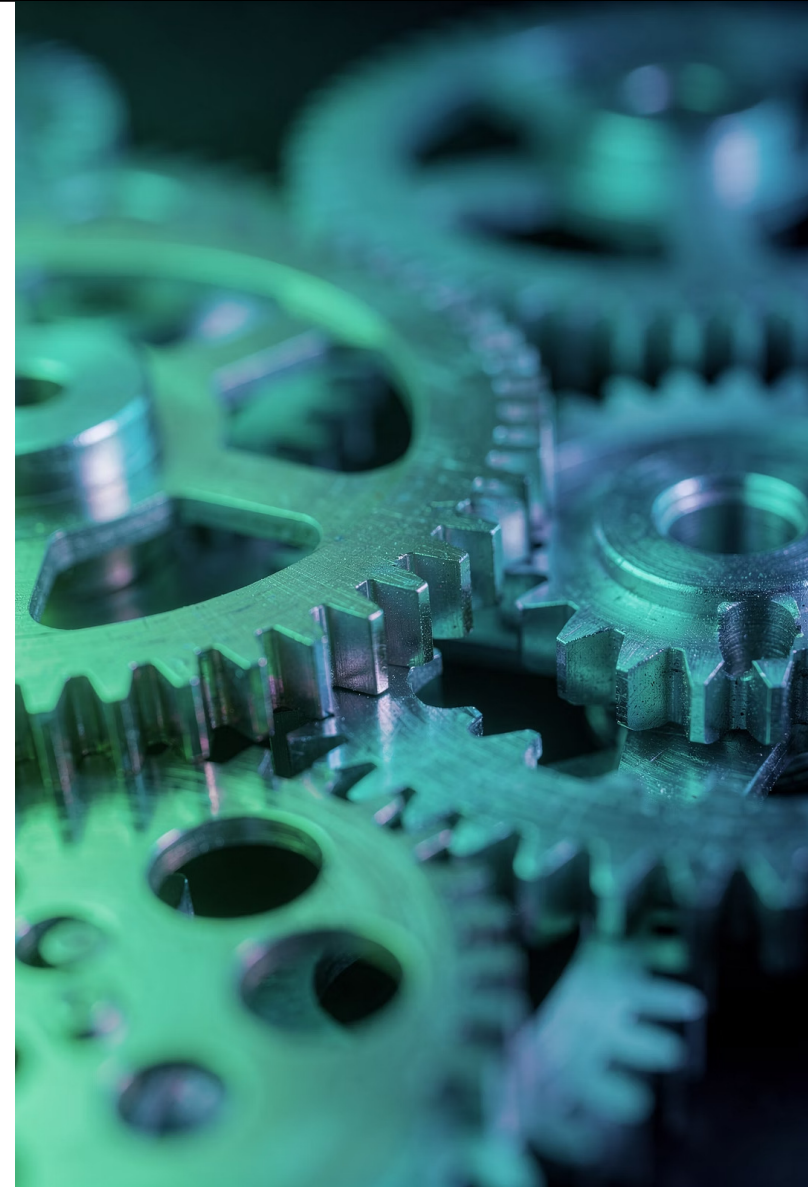
Permite alcanzar complejidades de tiempo altamente eficientes, a menudo $O(n \log n)$, lo que lo hace indispensable para grandes volúmenes de datos.

4 Análisis Riguroso

El rendimiento de estos algoritmos se evalúa mediante relaciones de recurrencia, cuya solución frecuentemente se aborda con el Teorema Maestro.

5 Aplicaciones Ubicuas

Es la base de algoritmos cruciales como la Búsqueda Binaria, Mergesort, Quicksort, y métodos para encontrar elementos máximos/mínimos.



Referencias

Para aquellos interesados en profundizar en el diseño y análisis de algoritmos "Divide y Vencerás", a continuación se presenta una selección de textos académicos fundamentales:

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- **Kleinberg, J., & Tardos, É.** (2006). *Algorithm Design*. Addison-Wesley.
- **Sedgewick, R., & Wayne, K.** (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- **Knuth, D. E.** (1998). *The Art of Computer Programming, Vol. 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.